Online PlatoSim

This is a document for the purposes of explaining the changes made in PlatoSim to include online capabilities within the tool. In the following the implantation, usage and reasoning behind the made decisions will be explained.

Contents

Purpose

PlatoSim in its original form was never intended to be something different than an offline tool to perform simulations with previously known input data. For the purposes of testing the FGS under real-time conditions PlatoSim had to be adapted to fulfill four additional tasks it was originally not designed to do:

1. Receiving jitter data during the simulation from a server
2. Receiving the position data of the used subfield from a server
3. Sending the created imagette data to a client as soon as it is created
4. Processing the data of as much as 30 stars in parallel

These tasks made it necessary to include means to tackle the needed threading and network communication requirements into the PlatoSim codebase. This had to be done with minimal changes to the original code in order to not change the usual functionalities. Online PlatoSim was always intended to be an optional feature.

Architecture

In the following the changes to the PlatoSim architecture are described. It is separated into the general additions to include the means to communicate with servers and clients over TCP connections and the steps taken to enable the conduction of multiple simulations at the same time.

Online Capabilities

To handle the multi-threading and TCP connection related tasks it was decided to use the ZeroMQ library. It is a high-performance asynchronous messaging library that doesn't need an extra message broker and generally facilitates the handling of all TCP- and inter-thread messaging. It supports a variety of messaging patterns, is easy to implement and well documented. ZeroMQ sockets are used for all communications with potential servers/clients.

Beside the functions to send and receive messages over sockets, some extra methods had to be implemented within the original codebase to discriminate whether an offline or online simulation must be conducted during runtime of the simulator.

The inclusion of the needed additional functionalities to PlatoSim aimed to be as non-invasive as possible to the existing code base at the time. The main changes were the setting up of an abstract factory pattern to create a suitable detector class instance dynamically, depending on whether the new online functionalities of PlatoSim are wanted for a simulation.

The other major change where the addition of another dynamic instance of the existing jitter factory to include the jitterFromNetwork functionality.

The last addition is a new variant of the hdf5 class, that is created dynamically if the functionalities sendImagettesToClient or getWindowPositionFromServer are activated in the used input.yaml file.

The difference to the original hdf5 file is, that no imagettes or other pixel maps are written into it, after the simulation is started. The reasons for this are:

- Data can't be overwritten easily within the hdf5 file, what makes changing the PSF map, or the throughput map during runtime problematically
- Online simulations are not determined and can theoretically go on indefinitely. The required space for the imagette data can add up quickly.
- The imagettes are sent to a client anyway and are no longer needed after an online simulation

Simulation parallelization

The last requirement is the parallel processing of more than one simulation at the same time with the same jitter data delivered over a TCP connection. This problem has been solved easily by just starting more than one simulation at the same time in different shells. The various ZMQ connection patterns used within Online PlatoSim make it possible that the different tasks can be fulfilled for an arbitrary number of simulation instances which are all connected to the same jitter- and windowPosition server and the same imagette client. See the table below for details

Connection pair	Used connection pattern
JitterServer – OnlinePlatoSim	PUBLISHER – SUBSCRIBER
WinPositionServer – OnlinePlatoSim	ROUTER – DEALER
ImagetteClient - OnlinePlatoSim	ROUTER – DEALER

For the jitter connection a PUB – SUB pattern was used, since the jitterServer doesn't need to know, which of the, or even how many clients are receiving data from the jitter server. The simulation instances just subscribe to the server and receive data without ever sending something themselves.

The connection to the window Position server uses a ROUTER-PATTERN, since an asynchronous data exchange Is needed here. If an Online PlatoSim instance is ready to start with the integration of light after its initialization phase, a request for a window position is send to the window position server. With this request the identity of the simulation is sent and the server sends a unique window position to each respective simulation instance.

The connection to the imagette client also uses the ROUTER-DEALER pattern. In this case only the Online PlatoSim instances are sending data to a client. The client can identify a specific instance by its identity which is also sent with each imagette. This is important to map the imagette data to the window position data.

TODO: When the code was written I falsely assumed that the automatically generated identity of the simulation sent to the window position server and to the imagette client are the same. This is not the case, since two different sockets are used, and I strongly suggest setting the identity to a specific unique value for each simulation instance. I recommend the name of the output hdf5 file, since

these should be unique anyway. This would make the mapping of the imagette data to the window position data significantly easier.

Since Online PlatoSim will be used in a real-time closed-loop testing environment it is necessary to check, whether the environment the Online PlatoSim instances are running in has the resources to deal with the computational load. PlatoSim in its original form can be very demanding in its hardware requirements and it is important to know what could negatively impact the time requirements of a simulation especially when more than one will be conducted parallel.

The most demanding task in every simulation is the application of the influences of the PSF. PlatoSim has two major possibilities to do so: Using a mapped PSF or using an analytic PSF. In general using an analytic PSF model is much faster than using a mapped PSF model, but limits the options in regards of using different kinds of PSF's. Both ways have different parameters that impact the simulation time.

The most important influencing parameters are:

subfieldSize (imagetteRows, imagetteCols)

jitterFrequency

used Subpixels

Mapped PSF	Analytic PSF
subfield size (imagetteRows, imagetteCols)	Jitter frequency
Number of subPixels used	Number of stars within the subfield

In the conducted tests I was able to run 30 simulations in parallel with a subfield size of 32 x 32 and 64 subPixels per pixel with a mapped approach on a contemporary Linux machine under real-time conditions. This is the worst case scenario for the planned closed-loop tests and shows, that the concept of Online PlatoSim is working.

Installation

Installing this variant of PlatoSim isn't any different from installing the usual PlatoSim. For convenience the zeroMQ library was included to the installation process so it should work out of the box, if the system is prepared for PlatoSim. Just execute the install.sh script within the PlatoSim main folder after downloading it from github.

For a more detailed overview on how to set up your system for the use of PlatoSim consult the official PlatoSim guide (http://ivs-kuleuven.github.io/PlatoSim3/DownloadUpdateBuild.html) or use the installation manual I wrote a couple of years ago (A guide to PlatoSim).

If changes have been made to the existing code base there is no necessity to install all libraries again, instead the project can be compiled by typing:

make ..

make clean

make -j 4

in a terminal from the PlatoSim/build folder.

Usage

This chapter describes how to run tests and simulations with Online PlatoSim and how data has to be presented to be used by it.

Running online simulations

To use the online functionalities of PlatoSim the following parameters within the input.yaml document that is used for the simulation have to be set up as described here:

Platform/JitterSource:	FromNetwork
ControlTcpConnection/SendImagettesToClients:	True
ControlTcpConnection/GetWindowPositionFromServer:	True
ControlTcpConnection/JitterServerAddress:	"the IP address of the socket jitter is sent to from the jitterServer"
ControlTcpConnection/WndowPositionServerAddress:	"the IP address of the socket window positions are sent to from the winPositionServer"
ControlTcpConnection/ImagetteClientAddress:	"the IP address of the socket the imagettes are to be sent from PlatoSim to a imagetteClient"

If these parameters were set a simulation can be started by typing:

./platosim <path_to_the_used_input_file> <path to the output hdf5 file>

within a terminal within the PlatoSim/build directory:

After that the PlatoSim instance sends a ping over its winPositionSocket to alert the potential winPostionServer, that it is ready to receive winPostion data. The simulation will then wait for the time set in the input.yaml file under ControlTcpConnection/WindowPositionSocketTimeout for the initial position of the subfield.

After an initial position was received the simulation will start and will wait for jitterSteps. Ths waiting period is set in the input.yaml file under ControlTcpConnection/JitterSocketTimeOut.

If the timestamp of a received jitterStep exceeds the simulations exposure time, the readout procedure is started and the imagette is finished and is sent to the imagetteClient.

A new exposure starts with a check for new window position data. After the initial window position received before the first jitter step, the value set under ControlTcpConnection/WindowPositionSocketTimeout doesn't matter anymore and is reduced to zero to avoid waiting periods during a running simulation. It is important to mention that if a new window position and size is received PlatoSim will calculate a new PSF convolution plan, if the used PSF model set within input.yaml is:

PSF/Model: MappedFromFile

or:

PSF/Model: MappedGaussian

This takes some additional time during the simulation that shouldn't impact the creation of the following imagette not too much unless the used hardware is not up to the task, or the new window size is unusually large.

The simulation will go on as long as PlatoSim receives valid jitterSteps within the timeframe set within the input.yaml file (ControlTcpConnection/JitterSocketTimeOut) or until an empty jitterString is sent.

If a stop condition was met, the simulation will finish the imagette it was working on under the premise that every missing jitter step until the end of the exposure time has the following values:

yaw = pitch = roll = 0.0

The last imagette is still send to the client before the simulation ends.

For the correct usage of PlatoSim in online mode a jitter and window position server as well as an imagette client is needed. This can be special tools like the Image Generator Wrapper or the

connectionTest application which is automatically installed alongside PlatoSim. This tool is self-explanatory. It only needs to be started within the PlatoSim/build folder from terminal by typing:

./connectionTest

After that the instructions in the terminal can be followed to, for example, start one or more platoSim instances and send jitter steps to them. Received imagettes will be displayed (but not saved). This tool is a mere help to understand the basic functionalities of online PlatoSim and will only work with the standard settings (the tcp addresses for example are hard coded within PlatoSim/connnectionTest.cpp). For a more useful, realistically approach a different tool is needed.

Used Protocols

Online PlatoSim communicates with its surrounding via a TCP connection using ZMQ sockets and expects its counterparts to be ZMQ sockets as well. ZMQ uses ZMQ strings to exchange messages. The different interfaces of PlatoSim are sending and receiving space separated int or double values converted to a string.

It is noteworthy that every message from Online PlatoSim is lead by a identification string which is autogenerated at the start of a connection of a socket and doesn't change over the course of a simulation.

Input:

All messages which are sent to an Online PlatoSim instance have to have the following properties to make sure, the simulation instance can cope with them correctly.

JitterStep:

A jitterStep is a space separated string of the jitter time step followed by the double values for yaw, pitch and roll in Euler angles.

double <jitterTimeStep> [s]

double <yaw> [arcsec]

double <pitch> [arcsec]

double <roll> [arcsec]

WindowPosition:

Online PlatoSim expects the window position values, which are all space separated integer values, in the following form:

int <subFieldColumns> [#]

int <subFieldRows> [#]

int <subFieldColumns> [#]

int <zeroPointRow> [#]

int <zeroPointRow> [#]

int <orientationAngle> [deg]

Output:

Imagette:

The imagette message sent is a string of space separated values lead by the imagette number, the rows of the imagette, the cols of the imagette followed by the imagette values in row-major.

int <imagetteNumber> [#]

int <imagetteRows> [#]

int <imagetteCols> [#]

int <ImagetteValue(row = 0)(col = 0)> [ADU]

int <ImagetteValue(row = 0)(col = 1)> [ADU]

…

int <ImagetteValue(row = 0)(col = imagetteCols)> [ADU]

int <ImagetteValue(row = 1)(col = 0)> [ADU]

int <ImagetteValue(row = 1)(col = 1)> [ADU]

…

int <ImagetteValue(row = 1)(col = imagetteCols)> [ADU]

…

…

int <ImagetteValue(row = imagetteRows)(col = imagetteCols)> [ADU]